

SQL Avanzado

Joins, Agrupaciones, Subconsultas y Vistas

Motor: SQLite · Entorno: Google Colab · Docente: Pilar Rocío Sayán Mejía

■ ¿Qué aprenderás en este manual?

- ✓ JOIN: INNER, LEFT, RIGHT y FULL OUTER — cuándo y por qué usar cada uno
- ✓ GROUP BY y HAVING — agrupar datos y filtrar grupos
- ✓ Subconsultas — consultas dentro de consultas
- ✓ Vistas (VIEW) — guardar consultas reutilizables
- ✓ Buenas prácticas: alias, legibilidad y comentarios en SQL
- ✓ Lab D3: Análisis de ventas completo con clientes, pedidos y productos

Contenido

- | | |
|----|---|
| 01 | Repaso rápido: la base de datos del Lab D3 |
| 02 | JOIN — unir tablas: INNER, LEFT, RIGHT, FULL OUTER |
| 03 | GROUP BY y HAVING — agrupar y filtrar grupos |
| 04 | Subconsultas — consultas dentro de consultas |
| 05 | Vistas (VIEW) — reutilizar consultas |
| 06 | Buenas prácticas: alias, legibilidad y comentarios |
| 07 | Lab D3 completo: Top 5 clientes, análisis de ventas |
| 08 | ¿Cuándo usar INNER JOIN vs LEFT JOIN? — Resumen final |

01 Base de datos del Lab D3: Clientes, Pedidos y Productos

Antes de escribir cualquier JOIN necesitamos entender el modelo de datos. El Lab D3 tiene tres tablas relacionadas que simulan el sistema de ventas de una empresa en Cusco. Copia y ejecuta este código en Google Colab para crear la base de datos completa:

Celda 1 — Conexión e importaciones

```
import sqlite3                                -- Módulo SQLite incluido en Colab

from tabulate import tabulate                 -- Para mostrar tablas bonitas

conn = sqlite3.connect('labd3.db')            -- Crea o abre el archivo labd3.db
cur = conn.cursor()                           -- Cursor: el lápiz que ejecuta SQL

def ver(consulta):                             -- Función auxiliar para ver resultados
    cur.execute(consulta)                     -- Ejecuta el SQL
    filas = cur.fetchall()                    -- Obtiene todos los resultados
    cols = [d[0] for d in cur.description]    -- Nombres de columnas
    print(tabulate(filas, headers=cols,
        tablefmt='grid'))                     -- Imprime tabla
    print(f'{len(filas)} fila(s)\n')          -- Cantidad de resultados
```

Celda 2 — Crear las tres tablas

```
-- ■■ TABLA CLIENTES ■■

CREATE TABLE IF NOT EXISTS clientes (

    id_cliente INTEGER PRIMARY KEY
    AUTOINCREMENT,                             -- PK: identificador único

    nombre TEXT NOT NULL,                       -- Nombre completo del cliente

    ciudad TEXT DEFAULT 'Cusco',                -- Ciudad del cliente

    email TEXT UNIQUE                           -- Email único por cliente

);

-- ■■ TABLA PRODUCTOS ■■

CREATE TABLE IF NOT EXISTS productos (
```

```

id_producto INTEGER PRIMARY KEY
AUTOINCREMENT,

nombre TEXT NOT NULL,                -- Nombre del producto

categoria TEXT NOT NULL,             -- Software, Hardware, Servicio...

precio REAL NOT NULL                 -- Precio unitario en soles

);

-- ■■ TABLA PEDIDOS ■■

CREATE TABLE IF NOT EXISTS pedidos (

id_pedido INTEGER PRIMARY KEY
AUTOINCREMENT,

fecha TEXT NOT NULL,                 -- Formato: 'YYYY-MM-DD'

cantidad INTEGER NOT NULL,           -- Unidades compradas

id_cliente INTEGER NOT NULL,         -- FK → clientes

id_producto INTEGER NOT NULL,        -- FK → productos

FOREIGN KEY (id_cliente) REFERENCES
clientes(id_cliente),

FOREIGN KEY (id_producto) REFERENCES
productos(id_producto)

);

conn.commit()                        -- Guarda los CREATE TABLE

```

Celda 3 — Insertar datos de ejemplo

```

-- Clientes (10 registros)

INSERT INTO clientes (nombre, ciudad,
email) VALUES

('María Quispe', 'Cusco',
'mquispe@mail.com'),           -- id=1

('Carlos Ttito', 'Cusco',
'cttito@mail.com'),           -- id=2

('Ana Mamani', 'Puno', 'amamani@mail.com'), -- id=3

('Pedro Huanca', 'Cusco',
'phuanca@mail.com'),          -- id=4

```

```

('Lucia Condori', 'Arequipa',
'lcondori@mail.com'),          -- id=5

('Jose Apaza', 'Cusco', 'japaza@mail.com'), -- id=6

('Sofia Ccopa', 'Lima', 'sccopa@mail.com'), -- id=7

('Roberto Huallpa', 'Cusco',
'rhualpa@mail.com'),          -- id=8

('Diana Flores', 'Puno',
'dflores@mail.com'),          -- id=9

('Marco Salas', 'Cusco', NULL); -- id=10, sin email

-- Productos (6 registros)

INSERT INTO productos (nombre, categoria,
precio) VALUES

('Licencia Office 365', 'Software',
450.00),                      -- id=1

('Laptop Dell i7', 'Hardware', 3200.00), -- id=2

('Soporte Técnico', 'Servicio', 250.00), -- id=3

('Antivirus Anual', 'Software', 180.00), -- id=4

('Disco Externo 2TB', 'Hardware', 320.00), -- id=5

('Capacitacion SQL', 'Servicio', 500.00); -- id=6

-- Pedidos (15 registros)

INSERT INTO pedidos (fecha, cantidad,
id_cliente, id_producto) VALUES

('2024-01-10', 2, 1, 1),      -- María compra 2 licencias Office

('2024-01-15', 1, 2, 2),      -- Carlos compra 1 laptop

('2024-01-20', 3, 1, 4),      -- María compra 3 antivirus

('2024-02-05', 1, 3, 3),      -- Ana pide soporte técnico

('2024-02-10', 2, 4, 1),      -- Pedro compra 2 licencias

('2024-02-18', 1, 5, 6),      -- Lucía: capacitación SQL

('2024-03-01', 1, 2, 5),      -- Carlos: disco externo

('2024-03-10', 4, 6, 4),      -- José: 4 antivirus

('2024-03-22', 1, 1, 2),      -- María: laptop

```

```

('2024-04-05', 2, 7, 3),          -- Sofía: 2 soportes
('2024-04-15', 1, 8, 1),          -- Roberto: licencia
('2024-05-01', 3, 4, 4),          -- Pedro: 3 antivirus
('2024-05-10', 1, 9, 6),          -- Diana: capacitación
('2024-05-20', 2, 1, 3),          -- María: 2 soportes
('2024-06-01', 1, 2, 6);          -- Carlos: capacitación SQL

conn.commit()                     -- Guarda todos los INSERT

print('Base de datos lista')

```

■ Diagrama de relaciones

clientes (id_cliente) ←■■■ pedidos (id_cliente) ■■■→ productos (id_producto)

Un cliente puede tener MUCHOS pedidos. Un producto puede aparecer en MUCHOS pedidos.

La tabla pedidos es la tabla intermedia que conecta clientes con productos.

El monto de cada pedido = pedidos.cantidad × productos.precio

02 JOIN — Unir tablas: INNER, LEFT, RIGHT, FULL OUTER

Un JOIN combina filas de dos o más tablas basándose en una columna común. Existen cuatro tipos principales. La diferencia está en QUÉ filas se incluyen cuando no hay coincidencia en una de las tablas.

Tipo de JOIN	¿Qué devuelve?	Filas sin match
INNER JOIN	Solo las filas que tienen coincidencia en AMBAS tablas	Se descartan
LEFT JOIN	Todas las filas de la tabla IZQUIERDA + coincidencias de la DERECHA	NULL en columnas derechas
RIGHT JOIN	Todas las filas de la tabla DERECHA + coincidencias de la IZQUIERDA	NULL en columnas izquierdas
FULL OUTER JOIN	Todas las filas de AMBAS tablas, haya o no coincidencia	NULL en ambos lados

2.1 INNER JOIN — Solo lo que coincide en ambas tablas

Es el JOIN más común. Solo devuelve filas donde existe coincidencia en las dos tablas. Si un cliente no tiene pedidos, NO aparece en el resultado.

```
-- Pedidos con nombre de cliente y nombre
de producto

ver(''

SELECT p.id_pedido AS Pedido,

p.fecha AS Fecha,

c.nombre AS Cliente,                -- c = alias de clientes

pr.nombre AS Producto,              -- pr = alias de productos

p.cantidad AS Qty,                  -- Cantidad comprada

pr.precio AS Precio_Unit,           -- Precio unitario

p.cantidad * pr.precio AS Monto_Total -- Calculamos el monto aquí mismo

FROM pedidos p                      -- Tabla principal: pedidos

INNER JOIN clientes c ON p.id_cliente = c.id_cliente -- Une con clientes

INNER JOIN productos pr ON p.id_producto = pr.id_producto -- Une con productos

ORDER BY p.fecha;                   -- Ordenado por fecha

''')
```

Pedid o	Fecha	Cliente	Producto	Qt y	Precio_Unit	Monto_Tota l
------------	-------	---------	----------	---------	-------------	-----------------

1	2024-01-10	María Quispe	Licencia Office 365	2	450.0	900.0
2	2024-01-15	Carlos Ttito	Laptop Dell i7	1	3200.0	3200.0
3	2024-01-20	María Quispe	Antivirus Anual	3	180.0	540.0
4	2024-02-05	Ana Mamani	Soporte Técnico	1	250.0	250.0
...	...	...	...	..	...	...

2.2 LEFT JOIN — Todos los de la izquierda, aunque no tengan pedidos

Devuelve TODOS los clientes, incluso los que no han hecho ningún pedido. Donde no hay pedido, las columnas de pedidos aparecen como NULL.

```
-- Ver TODOS los clientes, hayan comprado o no
ver(''

SELECT c.nombre AS Cliente,
c.ciudad AS Ciudad,
COUNT(p.id_pedido) AS Total_Pedidos,           -- COUNT ignora los NULL automáticamente
COALESCE(SUM(p.cantidad * pr.precio), 0) AS     -- COALESCE convierte NULL en 0
Total_Comprado
FROM clientes c                                -- Tabla IZQUIERDA: todos los clientes
LEFT JOIN pedidos p ON c.id_cliente =          -- LEFT: mantiene clientes sin pedidos
p.id_cliente
LEFT JOIN productos pr ON p.id_producto =
pr.id_producto
GROUP BY c.id_cliente, c.nombre, c.ciudad      -- Agrupa por cliente
ORDER BY Total_Comprado DESC;                  -- De mayor a menor compra
''')
```

Cliente	Ciudad	Total_Pedidos	Total_Comprado
María Quispe	Cusco	4	5240.0
Carlos Ttito	Cusco	3	4000.0
Pedro Huanca	Cusco	2	1440.0

...	...	...	...
Marco Salas	Cusco	0	0.0

■ ■ Marco Salas aparece con 0 pedidos

Con INNER JOIN, Marco Salas NO aparecería porque no tiene pedidos.

Con LEFT JOIN, SÍ aparece con Total_Pedidos=0 y Total_Comprado=0.

Esto es clave para reportes de clientes inactivos.

2.3 RIGHT JOIN — Todos los de la derecha

Devuelve TODOS los registros de la tabla derecha, haya o no coincidencia en la izquierda. En SQLite, RIGHT JOIN no está soportado directamente — se simula invirtiendo el LEFT JOIN:

```
-- RIGHT JOIN en SQLite: se simula con LEFT
JOIN invertido

-- Equivale a: pedidos RIGHT JOIN clientes

ver(''

SELECT c.nombre AS Cliente,

p.fecha AS Fecha_Pedido,

pr.nombre AS Producto

FROM clientes c                                -- Tabla derecha pasa a ser izquierda

LEFT JOIN pedidos p ON c.id_cliente =

p.id_cliente

LEFT JOIN productos pr ON p.id_producto =

pr.id_producto

ORDER BY c.nombre;

'')
```

2.4 FULL OUTER JOIN — Todo de ambas tablas

SQLite tampoco soporta FULL OUTER JOIN directamente. Se simula combinando un LEFT JOIN y un RIGHT JOIN (invertido) con UNION:

```
-- FULL OUTER JOIN simulado con UNION en
SQLite

ver(''
```



```

SELECT c.nombre AS Cliente, p.id_pedido AS
Pedido

FROM clientes c

LEFT JOIN pedidos p ON c.id_cliente =
p.id_cliente                                -- Todos los clientes

-- UNION combina los dos resultados sin
duplicados

UNION

SELECT c.nombre AS Cliente, p.id_pedido AS
Pedido

FROM pedidos p

LEFT JOIN clientes c ON p.id_cliente =
c.id_cliente                                -- Todos los pedidos

ORDER BY Cliente;

'''

```

■ Resumen: SQLite y los JOINS

✓ INNER JOIN → soportado nativo

✓ LEFT JOIN → soportado nativo

✗ RIGHT JOIN → no nativo, se simula con LEFT JOIN invertido

✗ FULL OUTER JOIN → no nativo, se simula con UNION de dos LEFT JOINS

03 GROUP BY y HAVING — Agrupar y filtrar grupos

GROUP BY agrupa filas que tienen el mismo valor en una columna, para aplicarles funciones de resumen (COUNT, SUM, AVG, MAX, MIN). HAVING filtra los grupos resultantes, igual que WHERE filtra filas individuales.

■ Diferencia clave: WHERE vs HAVING

WHERE filtra filas ANTES de agrupar — actúa sobre filas individuales.

HAVING filtra grupos DESPUÉS de agrupar — actúa sobre el resultado del GROUP BY.

Regla práctica: si usas una función de agregación en el filtro (SUM, COUNT...), usa HAVING.

3.1 Total de ventas por categoría de producto

```
ver(''  
  
SELECT pr.categoria AS Categoria,  
  
COUNT(p.id_pedido) AS Num_Pedidos,           -- Cuántos pedidos por categoría  
  
SUM(p.cantidad * pr.precio) AS  
Total_Ventas,                                -- Suma de montos  
  
ROUND(AVG(pr.precio), 2) AS Precio_Promedio   -- Precio promedio de productos  
  
FROM pedidos p  
  
INNER JOIN productos pr ON p.id_producto =  
pr.id_producto  
  
GROUP BY pr.categoria                        -- Agrupa por categoría  
  
ORDER BY Total_Ventas DESC;                 -- De mayor a menor venta  
  
''')
```

Categoría	Num_Pedidos	Total_Ventas	Precio_Promedio
Hardware	4	6720.0	1760.0
Software	7	3870.0	315.0
Servicio	4	2250.0	375.0

3.2 HAVING — Solo categorías con ventas mayores a S/ 3000

```
ver(''  
  
SELECT pr.categoria AS Categoria,
```

```

SUM(p.cantidad*pr.precio) AS Total_Ventas

FROM pedidos p

INNER JOIN productos pr ON p.id_producto =
pr.id_producto

GROUP BY pr.categoria

HAVING SUM(p.cantidad * pr.precio) > 3000      -- HAVING filtra DESPUÉS de agrupar

ORDER BY Total_Ventas DESC;

'''

```

Categoria	Total_Ventas
Hardware	6720.0
Software	3870.0

3.3 GROUP BY con múltiples columnas — Ventas por mes y ciudad

```

ver(''

SELECT SUBSTR(p.fecha, 1, 7) AS Mes,      -- SUBSTR extrae YYYY-MM de la fecha
c.ciudad AS Ciudad,
COUNT(p.id_pedido) AS Pedidos,
SUM(p.cantidad * pr.precio) AS Total

FROM pedidos p

INNER JOIN clientes c ON p.id_cliente =
c.id_cliente

INNER JOIN productos pr ON p.id_producto =
pr.id_producto

GROUP BY SUBSTR(p.fecha, 1, 7), c.ciudad  -- Agrupa por mes Y ciudad

ORDER BY Mes, Total DESC;

'''

```

04 Subconsultas — Consultas dentro de consultas

Una subconsulta es un SELECT dentro de otro SELECT. Se coloca entre paréntesis y puede aparecer en el WHERE, en el FROM o en el SELECT. Permiten responder preguntas complejas en un solo paso.

4.1 Subconsulta en WHERE — Clientes que compraron más del promedio

```
-- Paso 1: calcular el promedio de compra
por cliente

-- Paso 2: mostrar solo los que superan ese
promedio

ver(''

SELECT c.nombre AS Cliente,

SUM(p.cantidad * pr.precio) AS
Total_Comprado

FROM pedidos p

INNER JOIN clientes c ON p.id_cliente =
c.id_cliente

INNER JOIN productos pr ON p.id_producto =
pr.id_producto

GROUP BY c.id_cliente

HAVING SUM(p.cantidad * pr.precio) > (           -- HAVING compara con la subconsulta

SELECT AVG(total_cliente) FROM (                -- Subconsulta anidada

SELECT SUM(p2.cantidad * pr2.precio) AS
total_cliente

FROM pedidos p2

INNER JOIN productos pr2 ON p2.id_producto
= pr2.id_producto

GROUP BY p2.id_cliente

)

)

ORDER BY Total_Comprado DESC;

''')
```

4.2 Subconsulta en FROM — Crear una tabla temporal

La subconsulta en el FROM actúa como una tabla temporal. Se le da un alias y se puede usar como si fuera una tabla real:

```
ver(''  
  
SELECT resumen.Cliente,  
  
resumen.Total_Comprado,  
  
CASE -- CASE es como un if-else en SQL  
  
WHEN resumen.Total_Comprado >= 5000 THEN  
'VIP'  
  
WHEN resumen.Total_Comprado >= 2000 THEN  
'Frecuente'  
  
ELSE 'Ocasional'  
  
END AS Categoria_Cliente  
  
FROM ( -- Inicio de la subconsulta en FROM  
  
SELECT c.nombre AS Cliente,  
  
SUM(p.cantidad * pr.precio) AS  
Total_Comprado  
  
FROM pedidos p  
  
INNER JOIN clientes c ON p.id_cliente =  
c.id_cliente  
  
INNER JOIN productos pr ON p.id_producto =  
pr.id_producto  
  
GROUP BY c.id_cliente  
  
) AS resumen -- El alias 'resumen' es obligatorio  
  
ORDER BY Total_Comprado DESC;  
  
''')
```

Cliente	Total_Comprado	Categoria_Cliente
María Quispe	5240.0	VIP
Carlos Ttito	4000.0	Frecuente
Pedro Huanca	1440.0	Ocasional
Ana Mamani	250.0	Ocasional

4.3 Subconsulta en SELECT — Producto más vendido por cliente

```

ver(''

SELECT c.nombre AS Cliente,

(SELECT pr.nombre                                -- Subconsulta que devuelve 1 valor

FROM pedidos p2

INNER JOIN productos pr ON p2.id_producto =
pr.id_producto

WHERE p2.id_cliente = c.id_cliente              -- Conecta con el cliente externo

GROUP BY pr.id_producto

ORDER BY SUM(p2.cantidad) DESC                  -- El más comprado

LIMIT 1) AS Producto_Favorito                  -- Solo el primero

FROM clientes c

WHERE c.id_cliente IN (SELECT DISTINCT
id_cliente FROM pedidos);                      -- Solo clientes con pedidos

'')

```

05 Vistas (VIEW) — Guardar y reutilizar consultas

■ ¿Qué es una VIEW?

Una VIEW (vista) es una consulta SELECT guardada con un nombre en la base de datos.

No almacena datos: cada vez que la consultas, ejecuta el SELECT original.

Ventajas: simplifican consultas complejas, mejoran la legibilidad y se reutilizan.

Se usan igual que una tabla normal: SELECT * FROM nombre_vista.

5.1 Crear una vista de resumen de ventas por cliente

```
-- Crear la vista (solo se hace una vez)

cur.execute('''

CREATE VIEW IF NOT EXISTS v_ventas_cliente
AS                                -- Nombre de la vista

SELECT c.id_cliente,

c.nombre AS Cliente,

c.ciudad AS Ciudad,

COUNT(p.id_pedido) AS Total_Pedidos,

SUM(p.cantidad * pr.precio) AS
Total_Comprado,

MAX(p.fecha) AS Ultima_Compra

FROM clientes c

LEFT JOIN pedidos p ON c.id_cliente =
p.id_cliente                    -- LEFT: incluye clientes sin pedidos

LEFT JOIN productos pr ON p.id_producto =
pr.id_producto

GROUP BY c.id_cliente, c.nombre, c.ciudad

''')

conn.commit()

print('Vista creada')
```

5.2 Usar la vista — igual que una tabla

```
-- Ahora puedes consultar la vista
simplemente así:

ver('SELECT * FROM v_ventas_cliente ORDER
BY Total_Comprado DESC')

-- También puedes filtrar sobre la vista:

ver(''

SELECT Cliente, Ciudad, Total_Comprado

FROM v_ventas_cliente                -- Usamos la vista como tabla

WHERE Ciudad = 'Cusco'              -- Filtramos sobre la vista

AND Total_Pedidos > 0                -- Solo clientes activos

ORDER BY Total_Comprado DESC

LIMIT 5;                            -- Solo los 5 primeros

''')
```

5.3 Crear vista de productos más vendidos

```
cur.execute(''

CREATE VIEW IF NOT EXISTS
v_productos_ranking AS

SELECT pr.id_producto,

pr.nombre AS Producto,

pr.categoria AS Categoria,

pr.precio AS Precio_Unit,

COUNT(p.id_pedido) AS Veces_Vendido,

SUM(p.cantidad) AS Unidades_Total,

SUM(p.cantidad*pr.precio) AS Ingresos_Total

FROM productos pr

LEFT JOIN pedidos p ON pr.id_producto =
p.id_producto

GROUP BY pr.id_producto

ORDER BY Ingresos_Total DESC

''')
```



```
conn.commit()

-- Usar la vista:

ver('SELECT * FROM v_productos_ranking')
```

5.4 Eliminar una vista

```
cur.execute('DROP VIEW IF EXISTS
v_ventas_cliente')          -- Elimina la vista (no los datos)

conn.commit()
```

06 Buenas prácticas: alias, legibilidad y comentarios

Un buen SQL no es solo el que funciona — es el que otra persona puede leer y entender sin preguntar. Estas prácticas son las que usan los profesionales.

6.1 Alias: nombres cortos y descriptivos

Usa alias para acortar nombres de tablas y para renombrar columnas en el resultado:

```
-- ■ MAL: sin alias, difícil de leer

SELECT clientes.nombre, pedidos.fecha,
productos.precio

FROM clientes INNER JOIN pedidos ON
clientes.id_cliente = pedidos.id_cliente

INNER JOIN productos ON pedidos.id_producto
= productos.id_producto;

-- ✓ BIEN: con alias claros y columnas
renombradas

SELECT c.nombre AS Cliente,           -- AS renombra la columna
p.fecha AS Fecha_Compra,
pr.precio AS Precio_Soles

FROM clientes c                       -- c = alias corto de clientes

INNER JOIN pedidos p ON c.id_cliente =
p.id_cliente                         -- p = alias de pedidos

INNER JOIN productos pr ON p.id_producto =
pr.id_producto;                     -- pr = alias de productos
```

6.2 Comentarios en SQL

```
-- Esto es un comentario de una línea

/*
Esto es un comentario
de múltiples líneas.

Útil para describir el propósito de la
consulta.

*/
```

```

SELECT c.nombre, -- Nombre del cliente           -- Comentario al final de línea

SUM(p.cantidad * pr.precio) -- Monto total

FROM pedidos p

INNER JOIN clientes c ON p.id_cliente =
c.id_cliente

INNER JOIN productos pr ON p.id_producto =
pr.id_producto

GROUP BY c.id_cliente;

```

6.3 Formato y legibilidad

Práctica	■ Evitar	✓ Preferir
Palabras clave	select * from pedidos	SELECT * FROM pedidos
Una cláusula por línea	SELECT a FROM b WHERE c GROUP BY c	SELECT a\nFROM b\nWHERE c
Indentación en subconsultas	SELECT x FROM(SELECT y FROM z)	SELECT x FROM (\n SELECT y\n FROM z\n)
Nombrar columnas calculadas	SUM(cantidad * precio)	SUM(cantidad*precio) AS Total
Alias de tablas	INNER JOIN clientes ON clientes.id=...	INNER JOIN clientes c ON c.id=...

07 Lab D3 completo — Análisis de ventas con JOINS

■ Objetivo del Lab D3

Aplicar todos los conceptos del manual sobre la base de datos labd3.db.

Responder 5 preguntas de negocio usando JOINS, GROUP BY, HAVING, subconsultas y vistas.

Al final tendrás un reporte completo listo para presentar.

Consulta D3-1: TOP 5 clientes por monto total comprado

Esta es la consulta más importante del laboratorio. Usamos INNER JOIN de tres tablas, GROUP BY y LIMIT:

```
ver(''

/*

LAB D3 – Consulta 1

Top 5 clientes por monto total comprado

Técnica: INNER JOIN x2 + GROUP BY + ORDER
BY + LIMIT

*/

SELECT c.nombre AS Cliente,
       c.ciudad AS Ciudad,
       COUNT(p.id_pedido) AS Num_Pedidos,           -- Cuántas veces compró
       SUM(p.cantidad) AS Unidades_Total,           -- Total de unidades
       SUM(p.cantidad * pr.precio) AS Monto_Total,   -- Monto en soles
       MAX(p.fecha) AS Ultima_Compra                -- Fecha de última compra
FROM pedidos p                                     -- Tabla central
INNER JOIN clientes c ON p.id_cliente =             -- Une clientes
c.id_cliente
INNER JOIN productos pr ON p.id_producto =          -- Une productos
pr.id_producto
GROUP BY c.id_cliente, c.nombre, c.ciudad           -- Un grupo por cliente
ORDER BY Monto_Total DESC                           -- De mayor a menor monto
LIMIT 5;                                             -- Solo el top 5
```

```
''')
```

Cliente	Ciudad	Num_Pedidos	Unidades_Tota l	Monto_Total	Ultima_Compra
María Quispe	Cusco	4	7	5240.0	2024-05-20
Carlos Ttito	Cusco	3	3	4000.0	2024-06-01
Pedro Huanca	Cusco	2	5	1440.0	2024-05-01
José Apaza	Cusco	1	4	720.0	2024-03-10
Sofía Ccopa	Lima	1	2	500.0	2024-04-05

Consulta D3-2: Productos más vendidos por categoría

```
ver(''  
  
SELECT pr.categoria AS Categoria,  
pr.nombre AS Producto,  
SUM(p.cantidad) AS Unidades_Vendidas,  
SUM(p.cantidad*pr.precio) AS Ingresos  
FROM pedidos p  
  
INNER JOIN productos pr ON p.id_producto =  
pr.id_producto  
  
GROUP BY pr.categoria, pr.id_producto           -- Agrupa por categoría y producto  
  
ORDER BY pr.categoria, Ingresos DESC;  
  
''')
```

Consulta D3-3: Clientes sin pedidos (LEFT JOIN + IS NULL)

```
ver(''  
  
SELECT c.nombre AS Cliente_Inactivo,  
c.ciudad AS Ciudad,  
c.email AS Email  
FROM clientes c  
  
LEFT JOIN pedidos p ON c.id_cliente =  
p.id_cliente           -- LEFT: incluye todos los clientes  
  
WHERE p.id_pedido IS NULL;           -- NULL = sin ningún pedido  
  
''')
```

```
''')
```

Cliente_Inactivo	Ciudad	Email
Marco Salas	Cusco	NULL

Consulta D3-4: Análisis mensual de ventas

```
ver(''  
  
SELECT SUBSTR(p.fecha,1,7) AS Mes,           -- YYYY-MM  
  
COUNT(DISTINCT p.id_cliente) AS           -- Cuántos clientes distintos  
Clientes_Activos,  
  
COUNT(p.id_pedido) AS Num_Pedidos,  
  
SUM(p.cantidad*pr.precio) AS Ventas_Mes,  
  
ROUND(AVG(p.cantidad*pr.precio),2) AS      -- Gasto promedio por pedido  
Ticket_Promedio  
  
FROM pedidos p  
  
INNER JOIN productos pr ON p.id_producto =  
pr.id_producto  
  
GROUP BY SUBSTR(p.fecha,1,7)               -- Agrupa por mes  
  
ORDER BY Mes;  
  
''')
```

Consulta D3-5: Reporte final con vista

```
-- Usar la vista v_ventas_cliente para el  
reporte final  
  
ver(''  
  
SELECT Cliente,  
  
Ciudad,  
  
Total_Pedidos,  
  
Total_Comprado,  
  
Ultima_Compra,  
  
CASE                                     -- Clasificación por monto  
  
WHEN Total_Comprado >= 5000 THEN '■ VIP'
```

```
WHEN Total_Comprado >= 2000 THEN '■  
Frecuente'  
  
WHEN Total_Comprado > 0 THEN '■ Ocasional'  
  
ELSE '■ Sin compras'  
  
END AS Segmento  
  
FROM v_ventas_cliente          -- Usamos la vista  
  
ORDER BY Total_Comprado DESC;  
  
'')
```

08 ¿Cuándo usar INNER JOIN vs LEFT JOIN?

Pregunta de negocio	JOIN recomendado	¿Por qué?
¿Qué clientes han comprado?	INNER JOIN	Solo los que tienen pedidos
¿Todos los clientes, hayan comprado o no?	LEFT JOIN	Incluye clientes sin pedidos
¿Qué productos se han vendido alguna vez?	INNER JOIN	Solo los que aparecen en pedidos
¿Todos los productos, incluso los no vendidos?	LEFT JOIN	Muestra productos sin ventas
Reporte de ventas del mes	INNER JOIN	Solo transacciones reales
Buscar registros huérfanos (sin relación)	LEFT JOIN + IS NULL	Detecta datos sin coincidencia
Comparar listas de dos tablas	FULL OUTER JOIN	Ver diferencias entre tablas

■ Pregunta de reflexión para el laboratorio

¿Por qué en la Consulta D3-1 usamos INNER JOIN y no LEFT JOIN?

→ Porque solo queremos clientes QUE SÍ compraron. Un cliente sin pedidos no puede tener monto total.

¿Y si quisiéramos ver también los clientes que nunca compraron en el ranking?

→ Usaríamos LEFT JOIN y COALESCE(SUM(...), 0) para que aparezcan con S/ 0.00

Resumen de comandos avanzados del manual

Comando	¿Para qué sirve?	Ejemplo
INNER JOIN	Solo filas con coincidencia en ambas tablas	JOIN pedidos p ON c.id=p.id_c
LEFT JOIN	Todas las de la izquierda + coincidencias	LEFT JOIN pedidos p ON ...
UNION	Combina resultados de dos SELECT	SELECT a UNION SELECT b
GROUP BY	Agrupar filas para funciones de resumen	GROUP BY ciudad
HAVING	Filtra grupos (después de GROUP BY)	HAVING SUM(monto) > 1000
Subconsulta WHERE	Filtra con el resultado de otro SELECT	WHERE id IN (SELECT ...)
Subconsulta FROM	Usa un SELECT como tabla temporal	FROM (SELECT ...) AS t
CASE WHEN	Condición dentro de SELECT	CASE WHEN x>5 THEN 'A'
CREATE VIEW	Guarda una consulta con nombre reutilizable	CREATE VIEW v AS SELECT ...
COALESCE(x, 0)	Reemplaza NULL por un valor por defecto	COALESCE(SUM(m), 0)

SUBSTR(fecha, 1, 7)	Extrae parte de un texto (año-mes de una fecha)	SUBSTR('2024-05-10', 1, 7)
ROUND(valor, n)	Redondea a n decimales	ROUND(AVG(precio), 2)